

Travel Planning Optimization System Employing Genetic Algorithms with Multiple Parameters

Iuliana-Elena Rusu*, Adrian Alexandrescu†

Faculty of Automatic Control and Computer Engineering,

Gheorghe Asachi Technical University of Iași

Iași, Romania

*iuliana-elena.rusu@student.tuiasi.ro

†adrian.alexandrescu@academic.tuiasi.ro, aalexandrescu@tuiasi.ro

Abstract—Many people around the world enjoy traveling as a popular leisure activity. The process of trip planning can be time-consuming, requiring travelers to choose between various options and make decisions based on their preferences and interests.

This paper addresses the tourist trip design problem by proposing a Travel Planning Optimization System (TPOS), which includes an architecture that employs genetic algorithms to streamline travel planning. The paper's novelty lies in considering numerous travel factors, such as duration of visits, travel time, preferred types of locations, operating hours, destination popularity and the scheduling of places to eat in the daily program, into a single metric. This approach holds great promise in transforming travel planning by providing customized experiences that closely match the unique preferences and interests of each traveler, while using real location-related information in determining an efficient multiple-day itinerary.

Keywords—Travel planning, genetic algorithm, recommendation systems, tourist trip design problem, personalized itineraries.

I. INTRODUCTION

Traveling is something many of us dream about, but when it comes to actually planning a trip, it can feel overwhelming. With so many options and details to consider, where do you even start? In the last years, technology has revolutionized the way we plan and experience our trips [1]. Now, users can easily book flights, hotels, and activities online. Websites and apps make it simple to find what they need and book it fast. Technology can now recommend places to go and things to do based on what we like. It learns from our preferences and suggests things that we might enjoy. In short, technology has made traveling easier, safer, and more fun. One of the most noticeable ways AI has impacted the travel industry is through the development of chat bots, which can provide users with personalized recommendations, instant responses, booking assistance and travel itinerary management [2].

Considering these advancements, this paper aims to dig deeper into how technology can help plan trips better. The underlying problem is called the tourist trip design problem [3]. The solution introduced in this paper is based on this problem. Specifically, it looks into using genetic algorithms (GA) to make trip planning easier. Even though existing tech is handy, there are still issues with making travel plans efficiently to match what each person likes. By using genetic algorithms, this research suggests a way to handle these problems. The

algorithm takes into consideration factors such as how long do you want to stay somewhere, how long it takes to travel, what kind of places you like, when they're open, and what spots you really want to see. By analyzing all these things, the goal is to make personalized plans that fit each traveler's tastes. Also, the paper talks about how important it is to fine-tune the fitness model and parameters in the GA framework. Turning different travel factors into one score is not easy, but this paper tries to come up with a strong solution to make this process better in order to improve the traveler's experience.

The proposed novel architecture describes a platform in which the travelers can choose any city as a trip destination, select either the beginning and ending dates or the duration of the trip, choose types of locations they are interested in and any other points of interest, if any. Based on these factors, the users receive a personalized multi-day itinerary aligned with their choices. The platform also offers users the ability to sign-in, allowing them to save and share their trip plans.

The remainder of the paper is structured as follows. Section II presents the related work regarding genetic algorithms in travel industry. The architecture and design of the proposed solution are described in Section III. Experimental results are presented in Section IV, and the conclusions and future work regarding the proposed system are presented in Section V.

II. RELATED WORK

There are several well-known platforms on the market that value the power of recommendation systems and artificial intelligence to optimize travel planning experiences. Notably, the AI Trip Planner developed by TripAdvisor [4] stands out. This tool uses machine learning algorithms to analyze user preferences and behaviors, offering personalized recommendations for accommodations, attractions, and dining options. Unfortunately, the obtained results are not always satisfactory, it lacks flexibility, there is little customization, and it does not provide directions on how to get from one place to another.

Academic studies have looked into using genetic algorithms to make travel plans better. These studies focus on solving tricky problems like the Traveling Salesman Problem (TSP) in travel planning. In [5] authors describe the TSP characterized by the challenge of finding the shortest route that covers all cities exactly once and returns to the starting point. The main

issue regarding this proposed solution is that it cannot be solved in polynomial time for large problem sizes. The article proposes using genetic algorithms to minimize the runtime of TSP by parallelizing it on a distributed computing framework.

In [6] the authors employ genetic algorithms and pairwise preference elicitation for trip planning. They also consider the trip budget, which is a plus, but the pairwise elicitation can prove tedious and there is no consideration of existing ratings for objectives. Another paper handling GAs in travel planning is [7]. The authors schedule tourist objectives and also restaurants for lunch/dinner, like in our paper. Although they do not consider the ratings nor the opening times of the locations. This is also the case for the multi-day itinerary planning presented in [8]. Other papers involving genetic algorithms in trip planning are [9], in which the popularity of a location is given by the number of photos taken for that location, and [10], which uses a greedy approach to GAs. Both do not offer the same features as our proposed solution.

A research paper that considers time windows, restaurant selection and locations that are mandatory to be visited is [11]. The authors propose greedy and branch-and-cut Pareto-based techniques, but they do not handle location reviews and multiple days when scheduling. Authors in [12] came up with another way to plan trips for tourists. They used a famous problem called the Knapsack Problem (KP) to do this. In [13] it is proposed a Fuzzy Greedy Randomized Adaptive Search Procedure algorithm to address the Tourist Trip Design Problem with time windows. Unlike traditional algorithms, this approach integrates fuzzy logic, which allows for handling imprecisions in decision-making processes. It does this by balancing what is best to see with what the traveler wants, all while making sure it fits within the time available. There are also other algorithms that can produce a good solution to the trip planner problem such as A-star algorithm [14], the cockroach swarm optimization [15], geometric filtering and rounding algorithms [16], an iterated greedy algorithm [17] or a NSGA-II algorithm [18]. None of them consider all the preferences for trip planning that our solution allows.

III. PROPOSED SOLUTION

A. General Overview

The novel solution proposed in this paper, i.e., the Travel Planning Optimization System (TPOS), addresses the challenge of optimizing travel planning by using genetic algorithms. The primary focus of our solution is to provide users with personalized trip itineraries that align with their preferences and constraints.

Our solution comes from understanding the main issue with travel planning: different preferences and constraints, which can make planning a trip difficult. We want to make it easier by designing a solution to create travel plans automatically. Each plan is tailored for each traveler, taking into account what they like and need. By using genetic algorithms, our solution offers a multitude of benefits to users, it helps reduce stress from making too many decisions and makes sure travelers have the best time possible on their trips.

B. System Architecture Overview

The proposed architecture for the travel planning system is presented in Fig. 1. The solution architecture overview comprises five main components: a front-end server, a back-end server, a genetic algorithm, a cache solution and a database.

The Front-end server acts as the user interface so that users can view their trip itineraries, create new trips by selecting their preferences, and access various features offered by the system, such as editing the trips and sharing them with others.

The Back-end server handles the business logic and functionality of the system. It communicates with a relational database to store and retrieve user data, manages user authentication and authorization, and manages interactions with external systems for trip planning purposes.

The genetic algorithm is a key component of the system responsible for generating personalized trip itineraries. It is invoked by a service within the Back-end server when a user requests a new trip plan. The genetic algorithm utilizes data from Google Places API, such as location information and attraction ratings, to optimize the itinerary based on the user's preferences and constraints. A cache solution, i.e., Redis, is employed to increase efficiency by storing locally the fitness of already evaluated chromosomes and the required information from the API (it includes time-based cache invalidation).

C. Proposed Genetic Algorithm

The proposed algorithm is represented by the following:

- 1) Generate the initial population (100 chromosomes).
- 2) Compute the fitness of each chromosome.
- 3) While the stop condition (a fixed number of iterations or no significant change in the fitness for the last specified number of iterations) is not met, apply the operators:
 - a) Selection (tournament selection),
 - b) Crossover (adapted single-point crossover for variable-length chromosomes and chromosome trimming),
 - c) Mutation (swap mutation),
 - d) Evaluation (including elitism).
- 4) Output the best solution.

It is important to note that the solution presented in this paper involves the GA running separately for each day. At the beginning of each new day, the list of possible locations is updated, removing locations that were part of the best solution found in the previous days. By updating the list of possible locations for each day, the algorithm can adapt to the evolving context of the trip and customize the itinerary to the user's preferences and constraints. This approach leads to more diversity and an increased quality of the trip itinerary. The decision to run the GA separately for each day in the trip itinerary arises from the necessity to ensure that the algorithm can focus on optimizing each day's itinerary independently. Implementing the algorithm separately for each day simplifies the overall system design and implementation. It reduces complexity by breaking down the problem into smaller, more manageable components, making it easier to maintain and scale the solution over time.

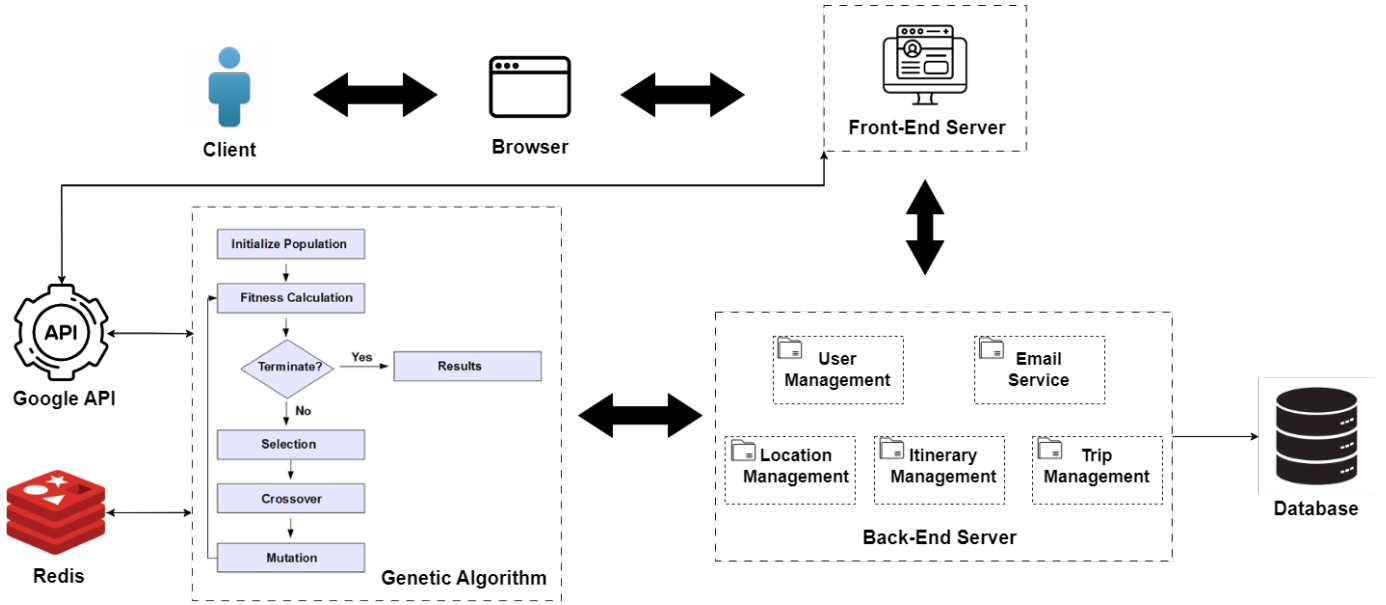


Fig. 1: Proposed architecture for the Travel Planning Optimization System

Separating the algorithm for each day promotes diversity in the generated solutions. Since each day is optimized independently, the algorithm has the opportunity to explore a wider range of possibilities and avoid converging to a suboptimal solution prematurely.

D. Obtaining and Modeling the Algorithm's Input Data

An important challenge was acquiring the necessary data. This included points of interest for cities worldwide, along with accompanying information such as photos, opening hours, latitude, and longitude coordinates. For this, the Google Places API is used because it also provided each location's opening hours, albeit through a more complex retrieval process than for the other features.

To initialize the population, firstly, location data is retrieved from the Google API or Redis and stored in a list. Random individuals are then generated. Each individual (chromosome) is an array of genes. The gene value is an integer representing the location index, which corresponds to the ordering in the initial list of locations obtained from the API. Fig. 2 represents two individuals with a different number of genes; there are no duplicate genes in a single chromosome, which means a point of interest can not be revisited.

K1	41	5	32	27	
K2	16	21	8	41	16

Fig. 2: An illustration of two chromosomes, K1 and K2, each with a different number of genes.

Taking data available in Google Places API is a challenge due to the large volume of data, specifically for large cities. To cope with this, an attempt was made to assign a score (or probability) to each location based on the number of reviews received for that location. In the population initialization phase, locations with higher scores were intended to appear with a higher probability. However, this approach led to the

domination of individuals with chromosomes of the 'must-see' type, those locations always having the most reviews, neglecting the user's other preferences. To address this issue, a solution was proposed to calculate a score for each location relative to other locations in the same category. This ensures fair comparison among locations of the same type, preventing bias towards specific types of locations. The score - $score(L)$ for location L can be calculated using the following formula:

$$score(L) = \frac{user_ratings_total(L)}{max_reviews} \quad (1)$$

where $user_ratings_total(L)$ represent the total number of user ratings received for location and $max_reviews$ represent the maximum number of user ratings received among all locations in the same category as L .

Fig. 3 illustrates the calculation of scores for actual points of interest in Paris, France. It demonstrates how locations of varying types and review counts can receive the same score, ensuring equal probability of appearing in solutions. This ensures no category of location is overlooked.

	Location	Reviews	Score
Type: Must-See Attractions Highest number of reviews: 394927	Tour Eiffel	394927	1
	Louvre Museum	312479	0.791
	Arc de Triomphe	242795	0.614
	Musée d'Orsay	90588	0.229
Type: Parks Highest number of reviews: 197953	Champ de Mars	197953	1
	Trocadéro Gardens	134878	0.681
	Jardin du Luxembourg	107325	0.271
	Parc Monceau	19697	0.099

Fig. 3: Real Points of Interest in Paris, France and the number of reviews along with the corresponding score

To vary the number of locations to visit each day based on the available time, individuals of length 6 are generated.

The algorithm checks the arrival time and trims the individual until around 4:00 PM, considering the day starts at 9:00 AM. This ensures that the itinerary for each day reflects the available time for sightseeing. Additionally, each location in the itinerary receives a predefined visit time (in minutes) based on its type: Museum 120, Park 60, Shopping 120, Zoo 180.

The algorithm aims to include two restaurants per day, in addition to the user's preferences, and the fitness function penalizes solutions that do not include exactly two restaurants arranged correctly within the chromosome, i.e., around lunch and dinner. However, due to the random generation of chromosomes, it was observed that many solutions did not meet this criterion. Even among the solutions that did contain two restaurants, they often fell short in terms of other criteria such as travel time, overall itinerary quality, and user preferences. To address this issue, when generating random individuals, all types of locations chosen by the user are added, except for restaurants. After trimming the individual, two restaurants are added: one in the middle of the individual (for lunch) and one at the end (for dinner). For the lunchtime restaurant, its position is calculated to minimize the total distance between the restaurant and the preceding and succeeding locations. Similarly, for the dinner restaurant, the goal is to minimize the distance between the last location and the restaurant.

Even after this process, crossover and mutation methods might change the order of the genes in chromosomes, so we still have to apply penalties regarding restaurants in the fitness function. This ensures that the desired criteria of having exactly two restaurants positioned correctly for lunch and dinner are maintained throughout the evolution process.

E. Determining the Quality of a Solution

The fitness function is a crucial component of genetic algorithms as it represents a metric that evaluates the quality of potential solutions. Combining all the factors and constraints of the problem into a single numerical value, as required by the fitness function, posed a significant challenge. In our implementation, the fitness of a candidate solution is determined by factors such as the total distance traveled, waiting time, closure penalties, alignment to user preferences, and ratings of visited locations.

The user has the option to select various types of locations they wish to explore, including museums, zoos, parks, shopping malls, places of worship etc. They can assign a percentage value ranging from 0 to 100 to each type of location (L). The penalty for a chosen solution is determined by subtracting the chosen percentage for a location from 100. The penalty decreases as the chosen percentage increases. Each location in a solution incurs a penalty, meaning that a solution with fewer high-percentage points of interest will result in higher penalties overall.

$$penalty(L) = (100 - percentage(L)) \times 10 \quad (2)$$

The process begins by calculating the distance matrix. After retrieving the points of interest from the Google API, we obtain latitude and longitude coordinates for each location.

With this geographical information, we construct a distance matrix to calculate the distances between all pairs of locations. For each pair of locations, we compute the distance between them using the Haversine formula, which calculates the great-circle distance between two points given their latitude and longitude coordinates. Another approach is to use the Google Maps API and consider the mode of transportation and the time of day. This would have been more accurate, but it has limitations regarding the number of queries on the API and the chosen approach is enough for our proof-of-concept. Finally, the computed distances are stored in the distance matrix for further use in the optimization process. The distance obtained is utilized as a penalty, where a greater distance corresponds to a higher penalty. The process continues by applying penalties based on several criteria. These include penalization for waiting before opening hours, penalties for exceeding closing hours or spending excessive time at a location beyond its closing time and penalties for violating user preferences or ratings.

For example, if a location is visited outside its operating hours, the solution receives a penalty. Additionally, penalties are assigned based on the deviation from user preferences and ratings. Locations that do not align with user preferences or have low ratings receive penalties. The next step is to sum up all the penalties (Δ) and use the result in the fitness function:

$$fitness = 1/(1 + \Delta) \quad (3)$$

An important consideration in computing the fitness value is the day of the trip. Since operating hours for most locations vary from day to day, we keep track of the specific day for which the chromosome candidates to being the optimal solution. This means that the same chromosome might have a different fitness depending on the day of the trip.

Users have two options: selecting the start and end dates of the trip or specifying the trip length in days. For the first option, the genetic algorithm is executed separately for each day of the trip. As for the second option, the approach is as follows: the algorithm is executed with the start day set to each day of the week, and then the solution with the highest fitness value is selected. For instance, for a 2-day trip, the algorithm runs for Monday and Tuesday, then for Tuesday and Wednesday, and so forth until Sunday to Monday. Once we determine the optimal start day for the trip, we identify the first future corresponding calendar date.

A special type of penalty is the one addressed to the must-see location chosen by the user, who can list several special locations he/she really wants to visit during the trip. However, there might not be enough time to see all of them. To help the user prioritize, we use a penalty system. We compare the number of places the algorithm has managed to include in the itinerary so far (α) with the number of special places that were not visited yet, divided by the number of days left in the trip (β). For the first day of the trip, solutions who have not included enough special places compared to what is left to see, receive a penalty to encourage more inclusion. This penalty is calculated based on the difference between the number of

special places left to see and the number we have already included. For the following days, we adjust α and β based on what has been visited already and how many days are left. If there are still more special places left to visit than days remaining, we continue to penalize the itinerary to ensure the user gets to see as many of his chosen locations as possible. If there are too many listed locations for the number of days of the trip, it is likely that not all of them will be included, but the penalties help prioritize including as many as possible.

Fig. 4 illustrates how penalties are applied to individuals for a 2-day trip with 6 specified locations provided by the user. On the first day, there are 3 chromosomes. The first chromosome contains 0 desired genes, resulting in the largest penalty. The other 2 chromosomes do not receive any penalty since they contain at least β desired genes. Among these, K3, with the most desired locations in the solution (4), is chosen as the best individual. For the second day, assuming that 4 out of the 6 points of interest were already included in the solution and there is still 1 day and 2 locations to include, a similar logic is applied. The best chromosome is selected, containing those 2 genes corresponding to the remaining locations.

		No. of must-see locations in chromosome - α	No. of unvisited must-see locations/ No. of remaining days - β	Penalty ($\beta - \alpha$)*1000
Day 1 No. of unvisited must-see locations: 6 No. of remaining days: 2	K1	0	3	3000
	K2	3	3	0
	K3	4	3	0
Day 2 No. of unvisited must-see locations: 2 No. of remaining days: 1	K4	0	2	2000
	K5	1	2	1000
	K6	2	2	0

Fig. 4: Example of penalties applied to different chromosomes for a 2-day trip with 6 locations mentioned by user

F. Selection, Crossover and Mutation

With each iteration, four steps are taken in order to improve the solution. The initial step involves the selection process, where chromosomes are chosen with a specific probability for utilization when applying the crossover and mutation operators. The chosen selection method is tournament selection, in which individuals from the population are randomly chosen to compete in a tournament. The winner of each tournament, i.e., the fittest individual among the two, is chosen as a parent for the next generation.

After selecting the two parents, we apply the single-point crossover with a 0.7 probability, which will generate a new offspring chromosome. In our context, the chromosomes may have different lengths due to trimming based on the time available for each solution. The crossover point is chosen within the range of the minimum length of the parent chromosomes to ensure valid crossover operations. Since revisiting the same location is undesirable, we need to ensure that the crossover operation does not result in chromosomes with repeated genes, which would lead to poor quality solutions. After randomly selecting the crossover point, we verify if the gene added to a new child already exists. If so, this gene is not added.

Mutation is a genetic operator that introduces diversity into the population by randomly altering individuals chromosomes.

We chose to implement the Swap Mutation, with a probability of 0.2, which means there's a 20% chance that any given individual's chromosome will undergo mutation. The mutation involves randomly swapping two genes (locations) within the chromosome. This method was chosen because it is simple and effective at introducing changes to the chromosome while maintaining its overall structure.

IV. EXPERIMENTAL RESULTS

The experiments were conducted for different locations around the world. The idea is that users select a location they want to visit (e.g., the city of Paris), inputs their preferences and the solution provides the daily itinerary. The system supports any location because it uses Google API to get the data. For the tests, the following percentages were used: must-see attractions: 38%, museums: 81%, parks: 79%, rest of the location types: 0%. The goal was to provide the best itinerary for a 4-day trip. We tracked the fitness evolution of the best chromosome across iterations for each day, showcasing the algorithm's performance in optimizing trip itineraries. The algorithm was executed 100 times to ensure statistical robustness.

Fig. 5 illustrates the fitness evolution over 50 iterations for each day. Notably, as the algorithm operates with chromosomes representing solutions with a maximum of 6 genes (locations), we experimented with various numbers of generations and population dimensions. Through this exploration, we observed that despite the imposition of several penalties, the problem for a single day is not overly complex.

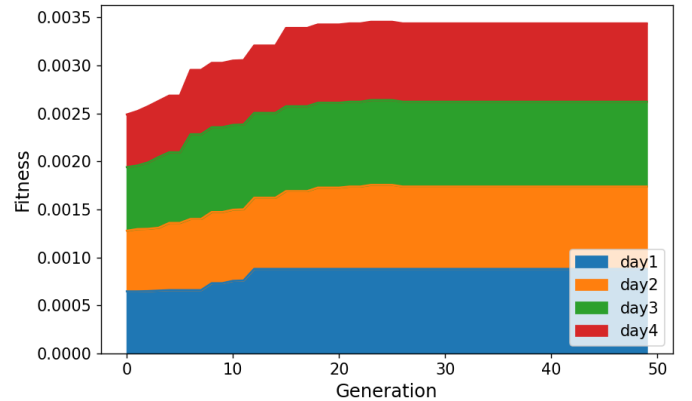


Fig. 5: The fitness evolution across iterations for a 4-day trip

Due to the preferences associated with certain locations (as determined by user-provided percentages), the algorithm tends to converge relatively quickly. This rapid convergence is evident in the fitness trends displayed in the graphic. One key reason for the rapid convergence of the genetic algorithm is the accent placed on creating a high-quality initial population. This was achieved through several measures, including the careful selection of locations and the implementation of scoring mechanisms. Firstly, by fine-tuning the process of generating random individuals, we ensured that the initial population was well-balanced and representative of diverse preferences. The inclusion of restaurants in strategic positions within the

itinerary, along with the optimization of their placement to minimize travel distances, contributed significantly to the effectiveness of the initial population.

Since the algorithm does not include locations from previous days in the solution for the current day, the available pool of locations reduces with each passing day. Consequently, the algorithm takes slightly longer to converge for each successive day compared to the initial day of the trip. Even though there are fewer places to choose from as the days go on, the fitness of each day's plan ends up getting really close to each other after a similar number of tries, although it might take a little longer for the later days. This shows that even with fewer options, the algorithm still figures out good plans without taking too many tries.

The fitness function imposes a penalty of 10000 for any gene within the chromosome that fails to meet a couple of specific conditions: (a) the gene must correspond to an hour and day when the location is open, (b) visit duration exceeds the remaining hours the location is open for the day. Another objective is to minimize the distance between any two points of interest by applying a penalty described in (4). This kind of penalty is applied to every pair of genes in an individual, except for the last gene, which has no companion.

$$\text{penalty}(L1) = \text{walking_distance}(L1, L2) \times 10000 \quad (4)$$

An itinerary example for a 4-day trip to Paris with must-see objectives and user preferences is shown in Fig. 6. Despite the promising results, it is essential to acknowledge the limitations, such as the simplified walking distance calculation and the dataset obtained from the Google API.

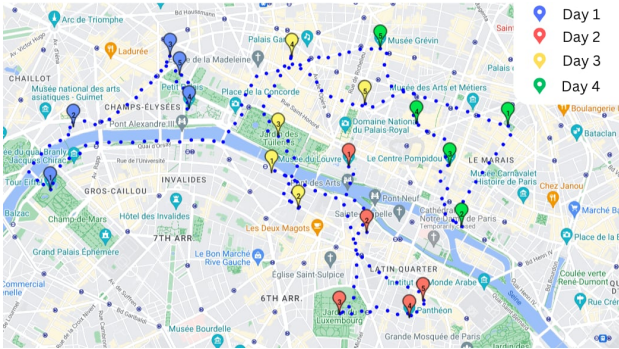


Fig. 6: Itinerary example for a 4-day trip to Paris

V. CONCLUSIONS

The presented research showcases a novel approach to handling the intricacies of trip planning through the utilization of genetic algorithms with a highly complex fitness function. By looking at how people plan their trips, we tried to make it easier to create personalized travel plans that match what each person likes. Despite the challenges addressed by the large variety of factors taken into consideration (e.g., location proximity, importance, opening hours, user preferences, multi-day planning) using real live data, our proposed Travel Planning Optimization System holds promise for addressing complex optimization problems in itinerary planning.

Future work can be done by adding scraper components to include other factors like which places are crowded at a

specific time, how is the weather at those locations when visiting, how busy are the roads, or what means of transport are better suited when traveling from one location to another. The system architecture even allows for a seamless update in the trip planning algorithm, e.g., a distributed island genetic algorithm or any other trip scheduling solution.

REFERENCES

- [1] J. Bulchand-Gidumal, "Impact of artificial intelligence in travel, tourism, and hospitality," in *Handbook of e-Tourism*, pp. 1943–1962, Springer, 2022.
- [2] D. Gursoy, Y. Li, and H. Song, "Chatgpt and the hospitality and tourism industry: an overview of current trends and future research directions," *Journal of Hospitality Marketing & Management*, vol. 32, no. 5, pp. 579–592, 2023.
- [3] P. Vansteenwegen, W. Souffriau, G. V. Berghe, and D. Van Oudheusden, "The city trip planner: an expert system for tourists," *Expert Systems with Applications*, vol. 38, no. 6, pp. 6540–6546, 2011.
- [4] Tripadvisor, LLC, "Tripadvisor launches ai-powered travel planning product," <https://tripadvisor.mediaroom.com/Tripadvisor-launches-AI-powered-travel-planning-product>, 2023. Last accessed 20 April 2024.
- [5] S. Chatterjee, C. Carrera, and L. A. Lynch, "Genetic algorithms and traveling salesman problems," *European journal of operational research*, vol. 93, no. 3, pp. 490–510, 1996.
- [6] N. N. Qomariyah and D. Kazakov, "A genetic-based pairwise trip planner recommender system," *Journal of big Data*, vol. 8, pp. 1–23, 2021.
- [7] B. S. Wibowo and M. Handayani, "A genetic algorithm for generating travel itinerary recommendation with restaurant selection," in *2018 IEEE International Conference on Industrial Engineering and Engineering Management (IEEM)*, pp. 427–431, IEEE, 2018.
- [8] T. Huang, Y.-J. Gong, Y.-H. Zhang, Z.-H. Zhan, and J. Zhang, "Automatic planning of multiple itineraries: A niching genetic evolution approach," *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 10, pp. 4225–4240, 2019.
- [9] P. Yochum, L. Chang, T. Gu, and M. Zhu, "An adaptive genetic algorithm for personalized itinerary planning," *IEEE Access*, vol. 8, pp. 88147–88157, 2020.
- [10] R. Paulavičius, L. Stripinis, S. Sutavičiūtė, D. Kočegarov, and E. Filatovas, "A novel greedy genetic algorithm-based personalized travel recommendation system," *Expert Systems with Applications*, vol. 230, p. 120580, 2023.
- [11] S. Choachaicharoenkul, D. Coit, and N. Wattanapongsakorn, "Multi-objective trip planning with solution ranking based on user preference and restaurant selection," *IEEE Access*, vol. 10, pp. 10688–10705, 2022.
- [12] M. Cosío-Léon, A. Martínez-Vargas, M. Lopez-Sanchez, and V. Silva-Rodríguez, "Genetic algorithm to maximize the tourist's satisfaction: An assessment of technology adoption for a tourist app," in *Optimization Methods for Product and System Design*, pp. 235–259, Springer, 2023.
- [13] J. Brito, A. Expósito-Márquez, and J. A. Moreno, "A fuzzy grasp algorithm for solving a tourist trip design problem," in *2017 IEEE International Conference on Fuzzy Systems (FUZZ-IEEE)*, pp. 1–6, IEEE, 2017.
- [14] Y. Zhou, X. Cheng, X. Lou, Z. Fang, and J. Ren, "Intelligent travel planning system based on a-star algorithm," in *2020 IEEE 4th Information Technology, Networking, Electronic and Automation Control Conference (ITNEC)*, vol. 1, pp. 426–430, IEEE, 2020.
- [15] J. Kwiecień and M. Pasieka, "Cockroach swarm optimization algorithm for travel planning," *Entropy*, vol. 19, no. 5, p. 213, 2017.
- [16] S. Edelkamp, S. Jabbar, and T. Willhalm, "Geometric travel planning," *IEEE Transactions on Intelligent Transportation Systems*, vol. 6, no. 1, pp. 5–16, 2005.
- [17] X. Wang, S. Wang, L. Wang, H. Zheng, J. Hao, R. He, and Z. Sun, "An effective iterated greedy algorithm for online route planning problem," in *2020 IEEE Congress on evolutionary computation (CEC)*, pp. 1–8, IEEE, 2020.
- [18] B. A. Beirigo and A. G. dos Santos, "Application of nsga-ii framework to the travel planning problem using real-world travel data," in *2016 IEEE Congress on Evolutionary Computation*, pp. 746–753, IEEE, 2016.